

포트폴리오 & 참고자료

DOCTRINE · 교리 — 설계 여정 · 엔지니어링 원칙 · 로드맵

"프롬프트 RTS"에서 "AI 사고방식을 키우는 게임"까지.

이 문서는 결과물이 아니라 **과정과 판단**을 담는다. 하나의 질문 — “왜 LLM이 아니면 안 되는가?” — 이 어떻게 게임의 정체성을 만들었는지, 그리고 그 정체성이 본선에서 어디로 향하는지를 정리한다.

01 설계 여정 — 정체성은 어떻게 생겼나

이 프로젝트의 핵심은 기능이 아니라 정의의 진화였다. 매 단계는 하나의 질문에 대한 답이었다: “이 게임에서 LLM이 아니면 안 되는 이유는?”

v1	프롬프트로 지휘하는 RTS “프롬프트가 곧 빌드.” 자연어로 유닛을 조종한다. — 문제: LLM이 없어도 성립한다(UI만 AI).
v2	AI가 교리를 배우는 RTS AI = 조종기가 아니라 팀원. 승리 규칙을 저장해 다음 전투에 적용. — 진전: “왜 LLM인가”에 근접.
v3	플레이어의 자연어가 AI의 영구적 사고방식이 되는 게임 명령의 의도를 추론하고 원리로 일반화해 다음 전장에 스스로 적용. — 이 지점에서 LLM은 대체 불가능한 핵심 메커니즘이 된다.

진화의 방향은 일관됐다: “AI를 썼다”에서 “AI가 아니면 안 된다”로. 기능을 더한 게 아니라, 불필요한 것을 덜어내며 하나의 명제로 수렴했다 — “당신의 자연어 명령은 AI 사령관의 영구적인 교리로 진화한다.”

02 무엇을 넣지 않았는가 — 절제의 설계

예산의 완성도는 “얼마나 많이 넣었나”가 아니라 “무엇을 빼서 인과를 선명하게 했나”로 결정된다.

넣고 싶었던 것	대신 내린 판단
유닛 수 늘리기 (5v5 → 10v10)	사람이 아니라 상황 을 늘렸다 — 역할군·특성·예외. 유닛을 늘려도 AI가 배우는 건 그대로다.
전장 이벤트·적 적응·작전 회의	꼳다. 인과를 흐리면 “교리가 승리를 만들었다”가 안 보인다.
전투 중 실시간 마이크로 지휘	거부. 그건 그냥 RTS 조종기로의 회귀 — USP를 배신한다.
“Thinking…” 가짜 로딩 연출	거부. 없는 계산을 연기하는 것 = “UI만 AI”.
LLM 자기보고 확신도(Intent Confidence)	거부. 캘리브레이션 안 됨 + 실측 신뢰도를 오염시킨다.

03 엔지니어링 원칙 — 검증과 정직성

검증 우선 (Verification-first)

AI/LLM이 만든 것은 반드시 검증한다. 회귀 테스트 30/30(일반화·조건부 발동·예외 학습·교리 충돌 해소·없는 의도 안 지어내기·폴백의 한계·계보 실재성) + 헤드리스 시뮬레이션(각 30~40회).

연출이 아니라 사실 (No AI theater)

“AI가 생각하는 것처럼 보이는” 가짜 연출을 거부. Commander 별명·교리 계보·신뢰도는 모두 실제 데이터에서 유도된 것이다. 지어낸 것은 하나도 없다.

정직한 폴백 (Honest fallback)

키가 없을 때의 규칙 기반 동작을 LLM인 척 포장하지 않는다. 화면에 추론 백엔드를 항상 표시 (OPENAI · gpt-4o / LOCAL · 규칙 기반).

교리는 승패를 가르다 — 밸런스 검증 (명령 없이 각 30회)

사령관의 사고방식	S1	S2	S3	S4
교리 없음	100%	100%	0%	100%
교리 (예외 없음)	100%	100%	100%	0%
교리 + 예외 (최종)	100%	100%	100%	100%

어떤 불완전한 사고방식도 4판을 모두 이기지 못한다. 학습의 최종 산물만이 모두 이긴다. 교리는 장식이 아니라 승패를 가르 는 판단이라는 것을, 밸런스가 강제한다.

04 개발 방식 — AI-native

제품의 AI(GPT-4o)와 별개로, 이 프로토타입은 Claude(Claude Code)와의 협업으로 개발되었다. 자연어로 요구를 전달 → 코드로 구현 → 헤드리스 시뮬레이터로 검증하는 루프를 반복했다. 여러 관점의 비판(“왜 LLM인가?”, “3분에 이해되는가?”, “기억에 남는가?”)을 설계에 직접 반영했고, 그 과정이 곧 제품의 정체성을 버려냈다.

스택: 순수 HTML / JavaScript / Canvas (의존성 0) · Node 내장 fetch 기반 LLM 프록시 · OpenAI GPT-4o (structured outputs) / Anthropic 어댑터 · localStorage 영속화. 빌드 도구·프레임워크 없음 — 브라우저만으로 실행된다.

05 로드맵 — 본선에서 증명할 것

예선에서 의도적으로 제외된 것들이 곧 다음 단계의 무기다. 하나의 핵심(교리 학습)을 완성했으니, 그 위에 층을 쌓는다.

- 본선 ① 사령관의 정체성 — Personality & Relationship**
학습된 교리에서 유도되는 별명을 넘어, 판단 스타일에 성격이 생기고 플레이어와 관계를 맺는다.
- 본선 ② 사령관의 공유 — Version & Sharing**
플레이어마다 다르게 자란 사령관을 저장·공유·대결. “내 사고방식 vs 네 사고방식.”
- 본선 ③ 집단 지성 — Multi-Agent Debate**
여러 사령관이 서로 다른 교리를 놓고 토론해 전략을 합의한다.
- 본선 ④ 기억과 교란 — Long-term Memory & Adversarial**
전투 히스토리 기반의 장기 기억, 그리고 적이 플레이어의 교리를 역이용·교란하는 방어 메타.

이 로드맵은 “아이디어”가 아니라 이미 검증된 핵심(자연어→의도→교리→적용) 위에 쌓이는 확장이다. 그래서 실현 가능하고, 그래서 이 팀은 본선에서 다음을 보여줄 수 있다.

06 참고 자료

소스코드 (GitHub)	github.com/cynkai/doctrine — 의존성 0, MIT
회귀 테스트	<code>node scripts/proof.js</code> — 30/30
게임 소개 및 설명 문서	개요 · 플레이 방법 · 실행 방법 (별도 제출)
AI 활용 기술 문서	왜 LLM인가 · 프롬프트 · 실측 (별도 제출)

This project does not use AI to replace the player.
It uses AI to remember, generalize, and evolve the player’s strategy.